

Employee Sales Analysis with PySpark

A line-by-line explanation of a complete PySpark ETL script — reading, cleaning, aggregating, ranking, and writing employee sales data — built on a real 250-row dataset.

[PySpark](#)

[Window Functions](#)

[script.py](#)

Table of Contents

01	Project Overview & Dataset
02	Setting Up the SparkSession
03	Defining an Explicit Schema
04	Reading the Raw CSV
05	Cleaning & Transforming the Data
06	Aggregations: Department & Region
07	Aggregation: Per-Employee Totals
08	Window Function: Ranking Top Performers
09	Monthly Sales Trend
10	Writing Results & Closing the Session
11	Running the Project

Project Overview & Dataset

This project uses PySpark to analyze an **employee sales dataset** containing 250 transaction records spread across three departments (Electronics, Clothing, Grocery) and three regions (Cairo, Alexandria, Giza), covering sales from January through September 2024.

Raw Columns

Column	Type in File	Description
emp_id	Integer	Unique employee identifier
name	String	Employee full name
department	String	Electronics, Clothing, or Grocery
region	String	Cairo, Alexandria, or Giza
sales_amount	Double	Value of the sale in local currency
sale_date	String (M/D/YYYY)	Date of the transaction, e.g. "1/5/2024"

What the Script Produces

- Total and average sales per **department**
- Total sales per **region**
- Total sales per **employee**
- The **top performer in each department**, using a window function
- A **month-by-month sales trend** across the whole company

Project Structure

```
FOLDER LAYOUT employee_sales_project/
├── data/
│   └── employee_sales.csv      # the raw input file
├── output/                    # Spark writes results here
│   ├── dept_summary/
│   ├── region_summary/
│   └── employee_summary/
```

```
|   ├── monthly_trend/
|   └── top_performers/
└── script.py                # the full PySpark job
```

Chapter Summary

- 250 rows of employee sales data across 3 departments and 3 regions.
- The script cleans the data, then produces five business-ready output tables.

Setting Up the SparkSession

Every PySpark program begins by creating a **SparkSession** — the single entry point used to read data, run SQL, and build DataFrames.

```
script.py # The entry point to any PySpark program.
spark = SparkSession.builder \
    .appName("EmployeeSalesAnalysis") \
    .master("local[*]") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")
```

Line-by-Line

- `SparkSession.builder` — starts a configuration builder for the session.
- `.appName("EmployeeSalesAnalysis")` — a human-readable name shown in the Spark UI and logs, useful for identifying this job among others.
- `.master("local[*]")` — tells Spark to run **locally**, using every available CPU core (the `*`) instead of connecting to a remote cluster manager like YARN or Kubernetes.
- `.getOrCreate()` — creates a new session, or reuses one if it already exists in this process.
- `setLogLevel("WARN")` — Spark is very verbose by default (INFO-level logs for every stage/task); this quiets it down to warnings and errors only, so the printed output stays readable.

TIP — LOCAL VS. CLUSTER MODE

`local[*]` is perfect for development and small datasets like this one. In production, the same script would run unchanged against a real cluster just by changing how the job is submitted (e.g. via `spark-submit --master yarn`) — the DataFrame code itself doesn't need to change.

Chapter Summary

- `SparkSession` is the required entry point for every PySpark script.
- `local[*]` runs Spark on the local machine using all CPU cores.
- Lowering the log level keeps console output focused on your own print statements.

Defining an Explicit Schema

Rather than letting Spark guess column types (`inferSchema=True`), the script defines the schema up front using `StructType` .

```
script.py sales_schema = StructType([
    StructField("emp_id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("department", StringType(), True),
    StructField("region", StringType(), True),
    StructField("sales_amount", DoubleType(), True),
    StructField("sale_date", StringType(), True),
])
```

Why Not Just Use `inferSchema=True`?

Explicit Schema	<code>inferSchema=True</code>
Spark reads the file once	Spark scans the file twice — once to guess types, once to load data
Types are guaranteed and predictable	Types can be guessed incorrectly (e.g. a numeric-looking string)
Slightly more code up front	Less code, but riskier on production data

Why `sale_date` Is a `StringType` (for now)

The raw dates look like `"1/5/2024"`, which isn't a format Spark's CSV reader parses automatically into a date. Reading it as a plain string first — then explicitly converting it in Step 5 with `to_date()` — avoids silent parsing failures (which would otherwise show up as null dates with no clear warning).

TIP — EACH `StructField`'S THIRD ARGUMENT

The `True` in each `StructField(..., True)` means the column is **nullable** — Spark will accept missing values there instead of failing the whole read.

Chapter Summary

- An explicit schema avoids a double-read of the file and guarantees column types.
- Dates are read as strings first, then parsed deliberately in a later step.

Reading the Raw CSV

```
script.py raw_df = spark.read.csv(
    "data/employee_sales.csv",
    header=True,
    schema=sales_schema
)

raw_df.show(5)
print(f"Total rows loaded: {raw_df.count()}")
```

Line-by-Line

- `spark.read.csv(...)` — the standard way to load a CSV file into a DataFrame.
- `header=True` — tells Spark the first row contains column names, not data.
- `schema=sales_schema` — applies the schema built in the previous chapter instead of guessing types.
- `.show(5)` — prints the first 5 rows in a readable table format, useful for sanity-checking that the load worked.
- `.count()` — triggers a full pass over the data to count rows. This is an **action** (it runs a Spark job immediately), unlike most DataFrame operations which are lazy transformations.

```
OUTPUT +-----+-----+-----+-----+-----+-----+
|emp_id|      name| department|      region|sales_amount|sale_date|
+-----+-----+-----+-----+-----+-----+
|   101|Ahmed Hassan|Electronics|      Cairo|      4500.75| 1/5/2024|
|   102|Sara Mostafa|  Clothing|Alexandria|      1200.0| 1/6/2024|
|   103| Omar Khaled|Electronics|      Giza|      3300.5| 1/7/2024|
+-----+-----+-----+-----+-----+-----+
Total rows loaded: 250
```

WARNING — LAZY EVALUATION

Reading a CSV does not actually load any data yet — Spark builds a query plan and waits. Nothing runs until an **action** like `.show()`, `.count()`, or `.write()` is called. This lets Spark optimize the whole chain of transformations before executing anything.

Chapter Summary

- `spark.read.csv()` loads the file using the schema we defined.
- Spark transformations are lazy; only actions like `.show()` trigger execution.

Cleaning & Transforming the Data

```
script.py clean_df = raw_df.withColumn(
    "sale_date", to_date(col("sale_date"), "M/d/yyyy")
)

clean_df = clean_df.withColumn(
    "sale_month", date_format(col("sale_date"), "yyyy-MM")
)

clean_df = clean_df.dropna(subset=["emp_id", "sales_amount", "sale_date"])

clean_df.cache()
```

Line-by-Line

- `withColumn("sale_date", to_date(...))` — overwrites the string `sale_date` column with a real **DateType**, parsed using the pattern `"M/d/yyyy"` (month, day, 4-digit year — matching `"1/5/2024"`).
- `withColumn("sale_month", date_format(...))` — derives a new `"yyyy-MM"` string column (e.g. `"2024-01"`), which is easier to group by than a full date when building the monthly trend later.
- `dropna(subset=[...])` — removes any row where the given columns are null, including dates that failed to parse (which `to_date()` silently turns into `null` rather than raising an error).
- `.cache()` — tells Spark to keep this DataFrame in memory after it's first computed. Because `clean_df` is reused across five different aggregations later in the script, caching avoids re-reading and re-parsing the CSV from scratch each time.

OUTPUT

emp_id	name	department	region	sales_amount	sale_date	sale_month
101	Ahmed Hassan	Electronics	Cairo	4500.75	2024-01-05	2024-01
102	Sara Mostafa	Clothing	Alexandria	1200.0	2024-01-06	2024-01

TIP — WHY CACHE MATTERS HERE

Without `.cache()`, every downstream `.groupBy()` in this script would re-trigger the CSV read and date parsing from the start, because Spark recomputes lazy transformations from scratch each time an action is called. Caching materializes `clean_df` once and reuses it.

Chapter Summary

- `to_date()` converts the text date into a real `DateTime` column.
- A derived `sale_month` column simplifies later monthly grouping.
- `.cache()` avoids recomputing the same cleaning steps repeatedly.

Aggregations: Department & Region

```
script.py dept_summary = clean_df.groupBy("department") \
    .agg(
        _round(_sum("sales_amount"), 2).alias("total_sales"),
        _round(avg("sales_amount"), 2).alias("avg_sale"),
        count("emp_id").alias("num_transactions")
    ) \
    .orderBy(col("total_sales").desc())
```

Line-by-Line

- `groupBy("department")` — groups all rows sharing the same department value.
- `.agg(...)` — computes one or more aggregate functions per group in a single pass.
- `_sum("sales_amount")` — Spark's `sum` is imported with an alias (`_sum`) to avoid overwriting Python's built-in `sum()` function.
- `_round(..., 2)` — rounds the result to 2 decimal places for readability.
- `count("emp_id")` — counts how many transaction rows fall into each group.
- `.orderBy(col("total_sales").desc())` — sorts departments from highest to lowest total sales.

```
OUTPUT - dept_summary +-----+-----+-----+-----+
| department|total_sales|avg_sale|num_transactions|
+-----+-----+-----+-----+
|Electronics|  425894.2| 5010.52|             85|
|  Clothing|  115516.55| 1391.77|             83|
|   Grocery|   60554.75|  738.47|             82|
+-----+-----+-----+-----+
```

The same pattern is reused for region:

```
script.py region_summary = clean_df.groupBy("region") \
    .agg(
        _round(_sum("sales_amount"), 2).alias("total_sales"),
        count("emp_id").alias("num_transactions")
    ) \
    .orderBy(col("total_sales").desc())
```

```
OUTPUT - region_summary +-----+-----+-----+-----+
|  region|total_sales|num_transactions|
+-----+-----+-----+-----+
```

	Cairo	312852.9	124
	Giza	148824.65	63
	Alexandria	140287.95	63
+-----+-----+-----+			

Chapter Summary

- `groupBy().agg()` is Spark's core pattern for computing per-group statistics.
- Electronics leads by a wide margin in total sales; Cairo leads by region.

Aggregation: Per-Employee Totals

```
script.py employee_summary = clean_df.groupBy("emp_id", "name", "department") \
    .agg(
        _round(_sum("sales_amount"), 2).alias("total_sales"),
        count("emp_id").alias("num_transactions")
    ) \
    .orderBy(col("total_sales").desc())
```

Why Group by Three Columns?

Grouping by `emp_id`, `name`, and `department` together (rather than just `emp_id`) keeps the readable columns (`name`, `department`) attached to the result. Since each employee has exactly one name and one department, grouping by all three produces the same groups as grouping by `emp_id` alone — but with friendlier output.

OUTPUT – employee_summary (top rows)

emp_id	name	department	total_sales	num_transactions
170	Dina Fathy	Electronics	6250.6	1
140	Amr Ibrahim	Electronics	6150.5	1
203	Mostafa Mahmoud	Electronics	6100.6	1

TIP — num_transactions IS 1 PER EMPLOYEE HERE

In this dataset, each `emp_id` happens to appear only once, so `num_transactions` is always 1. In a dataset where employees make repeat sales, this same query would correctly sum and count across all of their transactions.

Chapter Summary

- Grouping by multiple columns is a simple way to keep descriptive fields in the output.
- The same `groupBy().agg()` pattern scales from department-level to employee-level analysis.

Window Function: Ranking Top Performers

A **window function** computes a value across a group of rows related to the current row — without collapsing those rows into one, the way `groupBy()` does. Here, it's used to rank employees **within** each department.

```
script.py dept_window = Window.partitionBy("department") \
    .orderBy(col("total_sales").desc())

ranked_employees = employee_summary.withColumn(
    "rank_in_department", dense_rank().over(dept_window)
)

ranked_employees.filter(col("rank_in_department") == 1).show()
```

Line-by-Line

- `Window.partitionBy("department")` — defines the "window" as all rows sharing the same department, similar to `groupBy`, but rows stay individually visible.
- `.orderBy(col("total_sales").desc())` — within each department, rows are ordered from highest to lowest total sales before ranking.
- `dense_rank().over(dept_window)` — assigns rank 1, 2, 3... within each department's window, restarting the count for every new department. `dense_rank()` (versus `rank()`) leaves no gaps in the ranking sequence when there are ties.
- `.filter(col("rank_in_department") == 1)` — keeps only the single top-ranked employee per department.

OUTPUT — top performer per department

emp_id	name	department	total_sales	rank_in_department
127	Esraa Khaled	Clothing	1775.6	1
170	Dina Fathy	Electronics	6250.6	1
207	Mohamed Essam	Grocery	940.5	1

TIP — `dense_rank()` vs. `rank()` vs. `row_number()`

`rank()` leaves gaps after ties (1, 2, 2, 4); `dense_rank()` does not (1, 2, 2, 3); `row_number()` ignores ties entirely and assigns a unique number to every row (1, 2, 3, 4). Use `dense_rank()` when ties should share a rank without skipping numbers.

Chapter Summary

- Window functions rank or compute values across related rows without merging them.
- `partitionBy` defines the groups; `orderBy` defines the ranking order within each group.

Monthly Sales Trend

```
script.py monthly_trend = clean_df.groupBy("sale_month") \
    .agg(_round(_sum("sales_amount"), 2).alias("total_sales")) \
    .orderBy("sale_month")
```

This reuses the `sale_month` column created back in Chapter 5 (`"2024-01"` , `"2024-02"` , ...) to group all transactions by calendar month and sum their sales, giving a simple time-series view of company performance.

```
OUTPUT - monthly_trend +-----+-----+
|sale_month|total_sales|
+-----+-----+
| 2024-01| 61869.75|
| 2024-02| 68958.0|
| 2024-03| 72590.45|
| 2024-04| 72884.75|
| 2024-05| 78735.65|
| 2024-06| 71225.35|
| 2024-07| 74225.8|
| 2024-08| 73955.5|
| 2024-09| 27520.25|
+-----+-----+
```

TIP — WHY SEPTEMBER LOOKS LOW

September's total is noticeably lower simply because the dataset only contains data through early September — not because of an actual sales decline. Always check the date range of the underlying data before drawing conclusions from a trend's final period.

Chapter Summary

- Deriving a clean `yyyy-MM` column earlier makes time-based grouping trivial.
- Sales trended upward from January through May before leveling off.

Writing Results & Closing the Session

```
script.py dept_summary.write.mode("overwrite").parquet("output/dept_summary")
region_summary.write.mode("overwrite").parquet("output/region_summary")
employee_summary.write.mode("overwrite").parquet("output/employee_summary")
monthly_trend.write.mode("overwrite").parquet("output/monthly_trend")

ranked_employees.filter(col("rank_in_department") == 1) \
    .coalesce(1) \
    .write.mode("overwrite") \
    .option("header", True) \
    .csv("output/top_performers")

spark.stop()
```

Line-by-Line

- `.write.mode("overwrite")` — writes the DataFrame to disk, replacing any existing output at that path (safe to re-run the script repeatedly during development).
- `.parquet(path)` — most outputs are saved as **Parquet**, a columnar binary format that's compressed, fast to read back into Spark, and preserves exact data types.
- `.coalesce(1)` — before writing the top-performers result, this merges all of Spark's internal partitions into a single one, so the CSV output is one readable file instead of being split across many part-files.
- `.option("header", True).csv(path)` — writes that result as CSV with a header row, since a small human-readable file is more convenient here than Parquet.
- `spark.stop()` — shuts down the SparkSession and releases its resources. This should always be the last line of a standalone Spark script.

WARNING — `coalesce(1)` AT SCALE

Forcing everything into a single partition is fine for small results like a three-row top-performers table. Doing this on a genuinely large dataset would force all the data through one task, becoming a memory and performance bottleneck.

Chapter Summary

- Parquet is the default choice for structured, reusable Spark output.
- `coalesce(1)` is used selectively to produce a single readable CSV file.

- `spark.stop()` cleanly ends the session.

Running the Project

```
TERMINAL cd employee_sales_project
python3 script.py
```

Running the script prints each intermediate result to the console (as shown throughout this guide) and writes five result sets under `output/`:

Output Path	Format	Contents
output/dept_summary/	Parquet	Total/avg sales per department
output/region_summary/	Parquet	Total sales per region
output/employee_summary/	Parquet	Total sales per employee
output/monthly_trend/	Parquet	Total sales per calendar month
output/top_performers/	CSV	Top-ranked employee per department

Possible Next Steps

- Load the Parquet outputs into a notebook and chart the monthly trend.
- Add a bronze/silver/gold layered structure if this were to grow into a larger pipeline.
- Parameterize the input/output paths instead of hardcoding them, for reuse on new data.

— End of Walkthrough —